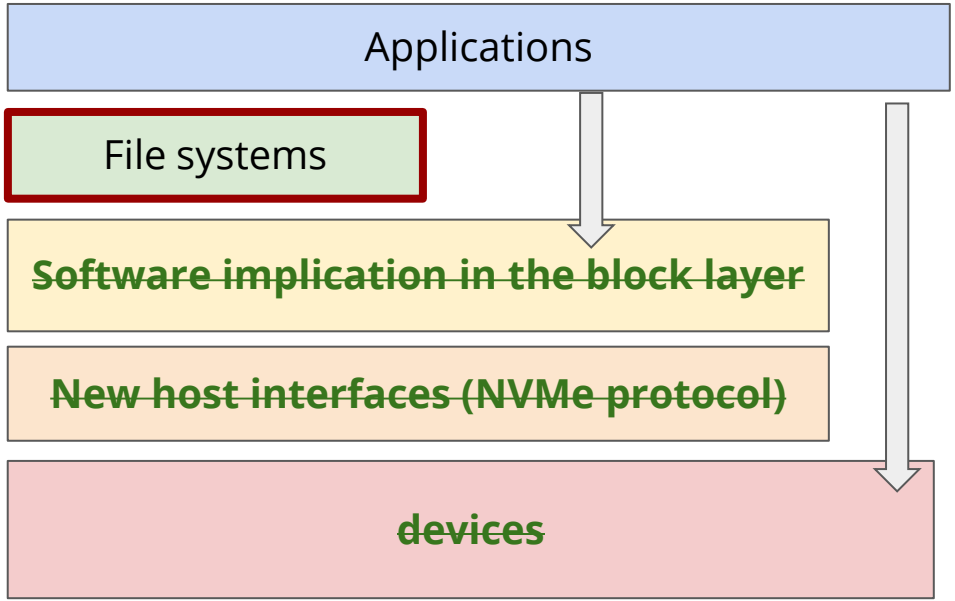
单机文件系统

崔立骁

Slides部分来源于Animesh Trivedi, Storage Systems (StoSys) XM_0092，以及清华大学存储系统课程

The layered approach in the lectures



Application for FS:

- Deepseek 3FS
- Distributed metadata management

Devices for FS:

- SSD
- NVM (persistent memory)

Recap: File System (FS)

FS is responsible for

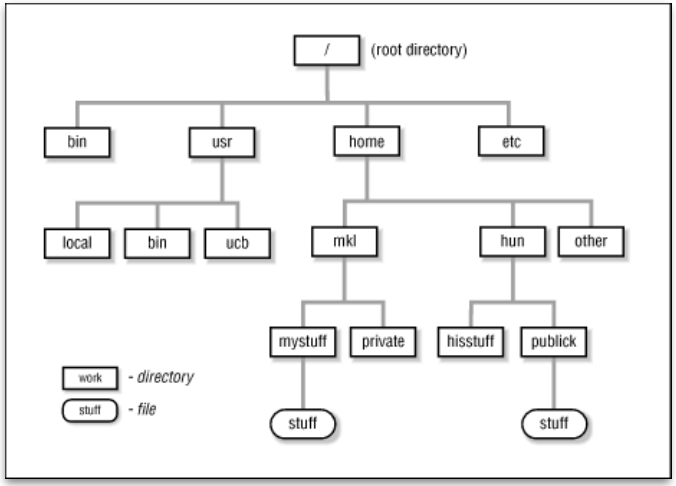
1. storing hierarchical directories and files on a flat disk;
2. translating user read/write to disk addresses

File systems have (1) data ; (2) metadata

1. **Data:** user data
2. **Metadata:** user and fs informations (name, creation time, storage location) etc.

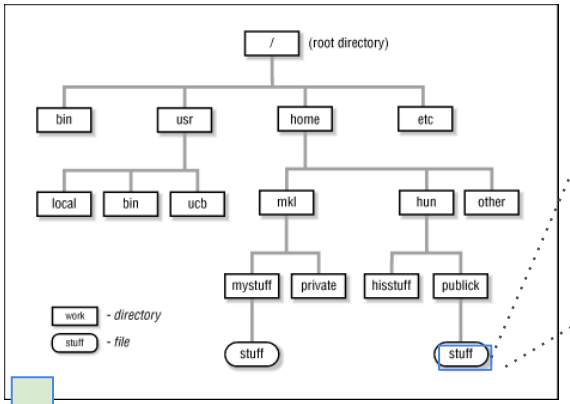
Important data structures: **inode** (stores file system metadata, and location of data)

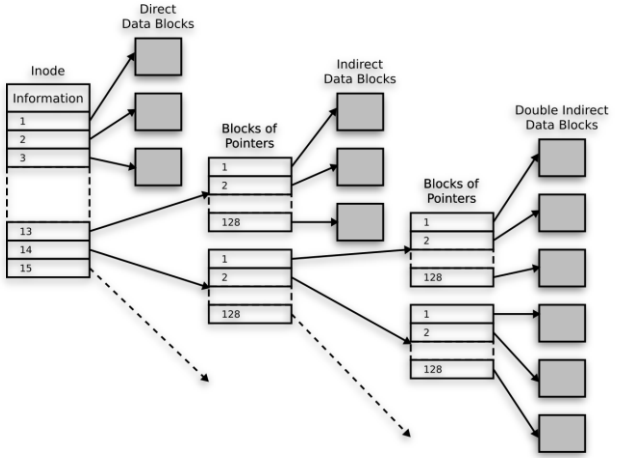
More: free bitmaps, extent maps, superblock, etc.

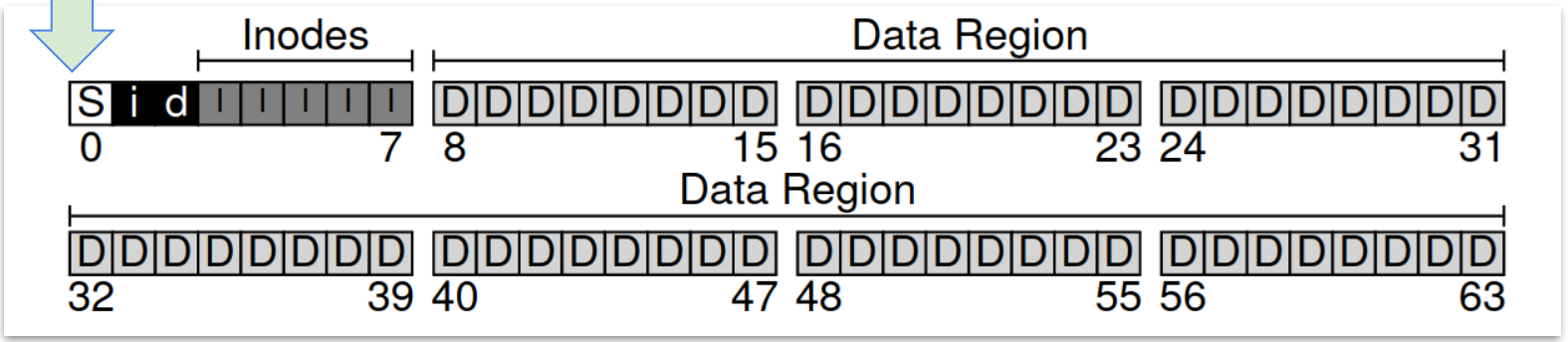


Background reading: chapter 40, File System Implementation, http://pages.cs.wisc.edu/~remzi/OSTEP/file-implementation.pdf

Recap: File System (FS)





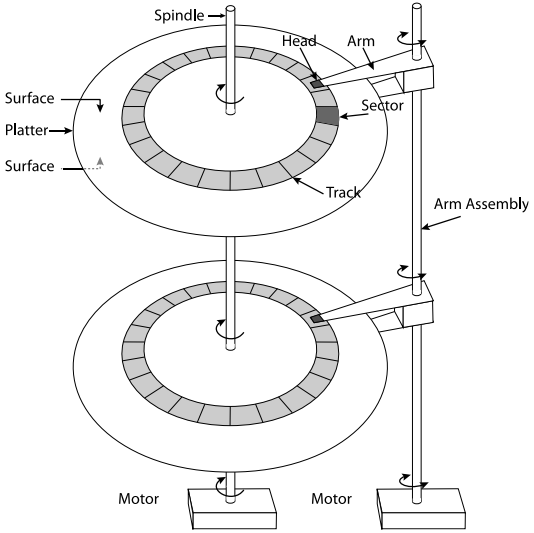


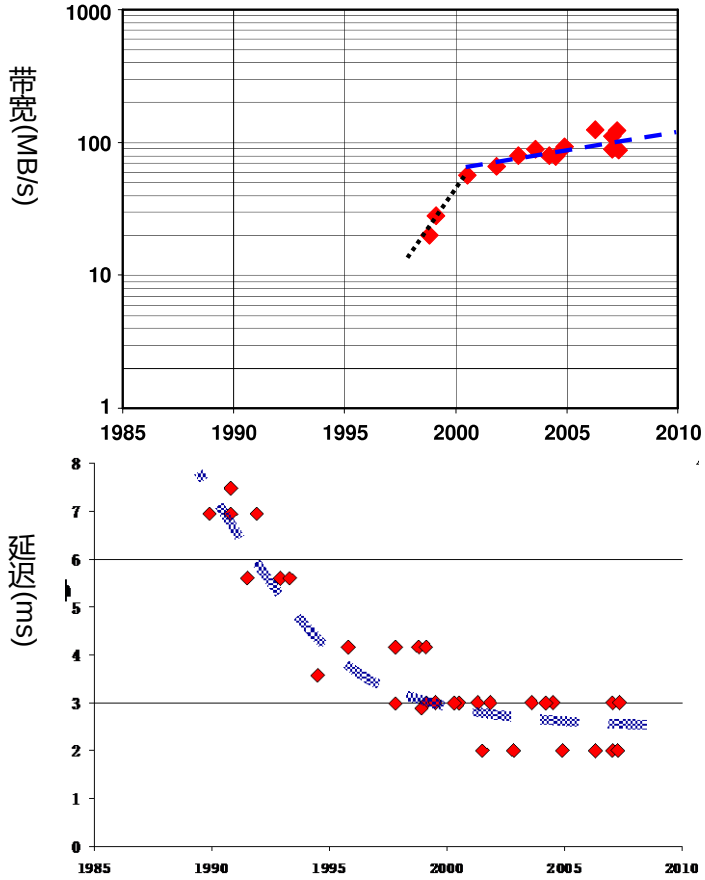
For example, ext2, https://wiki.osdev.org/Ext2 and https://piazza.com/class_profile/get_resource/il71xfllx3l16f/inz4wsb2m0w2oz

Flash SSD for File System

HDD的局限性

- 机械式外存 - 磁盘
- 性能局限性
  - 带宽、延迟





图片来源：FAST'10 Tutorial

HDD的局限性

- 能耗与体积局限性

Extrapolation to 2020

(at 70% CGR → need **2 GIOP/sec**)

磁盘存储：
5,000,000 块
超过 2,000 平方米
能耗 22 兆瓦

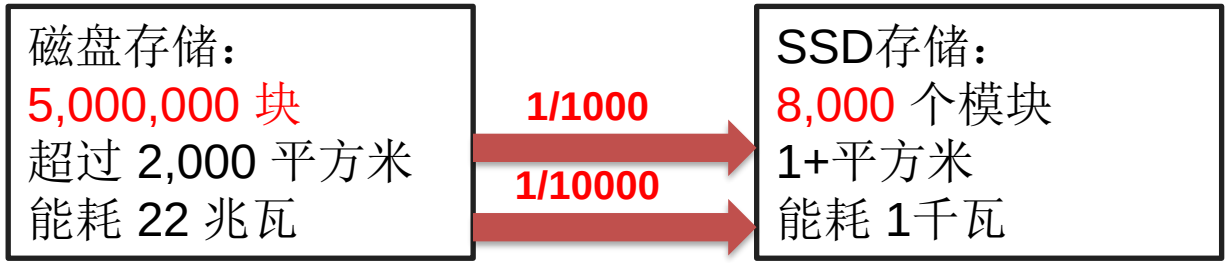
图片来源：Storage-class memory: the next storage system technology, IBM Journal

Flash SSD

- 读写性能：带宽与延迟

类型	设备型号	读带宽	写带宽	读延迟	写延迟
磁盘	Seagate Savvio	202	202	2.000	2.000
SATA 固态盘	Intel X25-E	250	170	0.075	0.085
PCIe 固态盘	FusionIO ioDrive Octal	6,000	4,400	0.030	0.030
		30X	20X	1/100	1/100

- 体积与能耗



Flash SSD

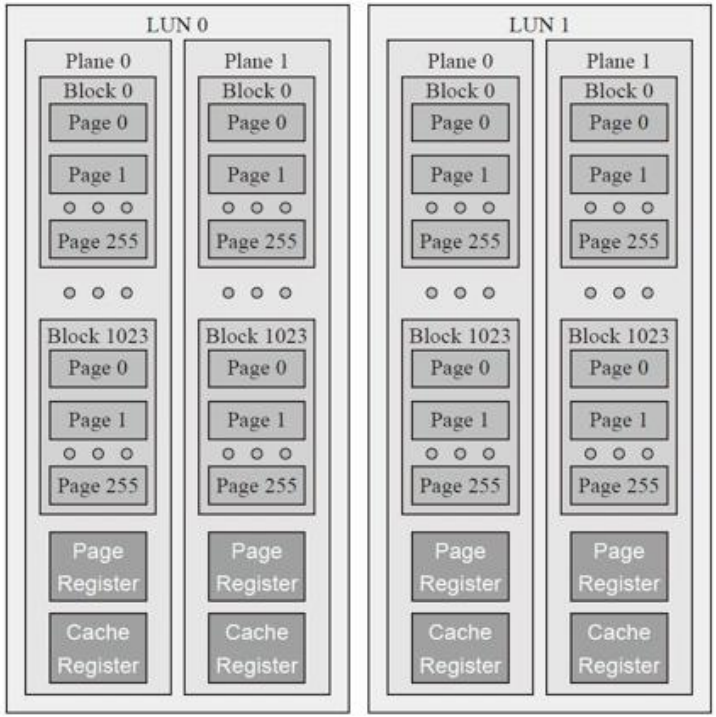


图3-7　闪存内部组织架构

Flash SSD

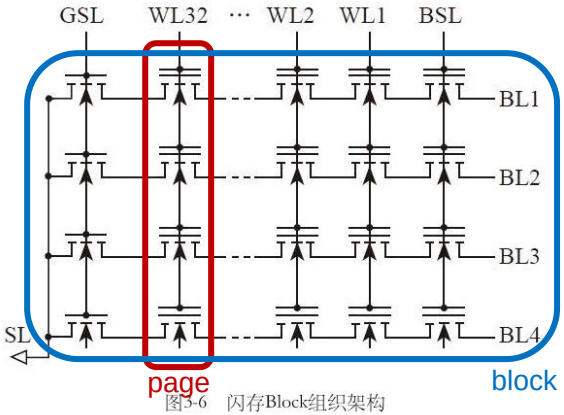


图3-6 闪存Block组织架构

- 最小读写单元为page。
- 擦除后的page只能写一次，要覆盖写page需要先擦除，擦除单位为block。
- 每个block存在最大擦除次数。

- 由于Block size大于page size，所以可能需要进行“读-修改-写操作”(Read-Modify-Write, RMW) 造成**写放大问题**。Over-provison(OP)和提前的垃圾回收(GC)可以减少SSD写放大!
- 由于每个block存在最大擦除次数，即使用寿命，所以需要**磨损均衡**。

图摘自《深入浅出SSD：固态存储核心技术、原理与实战》

SSD FTL

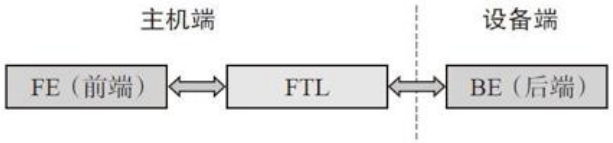


图4-1　FTL在主机端

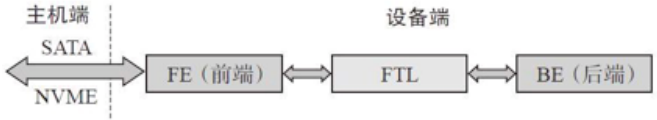


图4-2　FTL在设备端

一般实现于设备端，主要功能：

- 管理逻辑块-物理块的映射
- 负责垃圾回收(GC)和over-provison(OP)
- 磨损平衡

SSD FTL层的OP

	CH0			CH1			CH2			CH3		
	1	5	9	2	6	10	3	7	11	4	8	12
Block 0	13	17	21	14	18	22	15	19	23	16	20	24
	25	29	33	26	30	34	27	31	35	28	32	36
	37	41	45	38	42	46	39	43	47	40	44	48
Block 1	49	53	57	50	54	58	51	55	59	52	56	60
	61	65	69	62	66	70	63	67	71	64	68	72
	73	77	81	74	78	82	75	79	83	76	80	84
Block 2	85	89	93	86	90	94	87	91	95	88	92	96
	97	101	105	98	102	106	99	103	107	100	104	108
	109	113	117	110	114	118	111	115	119	112	116	120
Block 3	121	125	129	122	126	130	123	127	131	124	128	132
	133	137	141	134	138	142	135	139	143	136	140	144
	145	149	153	146	150	154	147	151	155	148	152	156
Block 4	157	161	165	158	162	166	159	163	167	160	164	168
	169	173	177	170	174	178	171	175	179	172	176	180
Block 5												

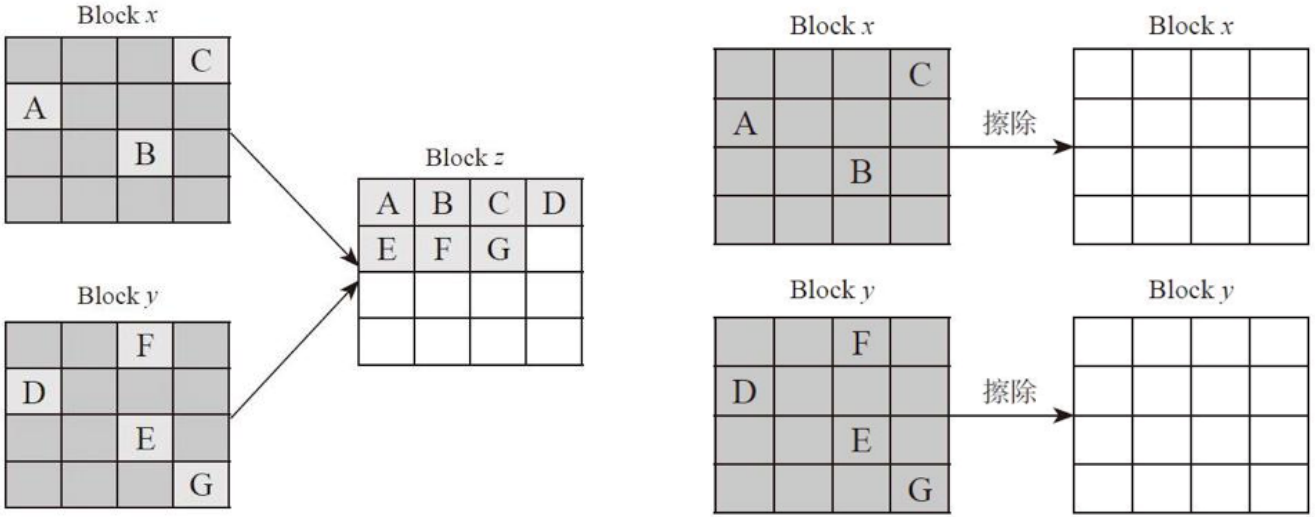
考虑覆盖写block 0 中的某一page：

1) Block 5不作OP， 暴露给主机：
需要读block 0中 9 个page到buffer，
修改其中1个page，擦除block 0，
再写入9页，造成了“写放大”。

2) Block 5 作OP，不暴露给主机：
可以直接写Block 5

图摘自《深入浅出SSD：固态存储核心技术、原理与实战》

SSD Garbage collection

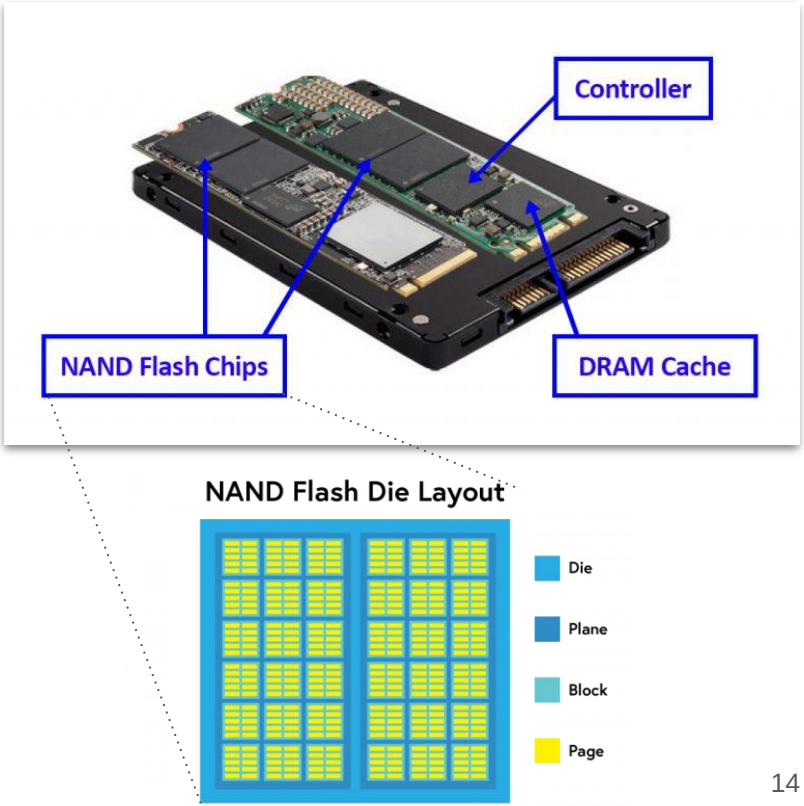


图摘自《深入浅出SSD：固态存储核心技术、原理与实战》

Why Do we really need a new file system?

NAND Flash SSD, even though “semantically” is like HDD (read/write sectors), internally it has:

- Mixed performance spectrum:
  - Very good sequential performance
  - Good random read performance
  - Poor random write performance
  - Very poor small random write performance
- FTL implementation
- GC interference
- Chip-die-plane parallelism



What happens if we just ignore it

Technically we can just run any file system. Sure it will work, but

1. Poor degraded performance
2. Unpredictable performance (*cloud providers do not like this!*)
3. Poor device lifetime

Bad things will happen :(Let's do our best try to avoid bad things.

- Immutable, sequential→ perfect for flash !

Interestingly enough …

The Design and Implementation of a Log-Structured File System

MENDEL ROSENBLUM and JOHN K. OUSTERHOUT
University of California at Berkeley

This paper presents a new technique for disk storage management called a *log-structured file system*. A log-structured file system writes all modifications to disk sequentially in a log-like structure, thereby speeding up both file writing and crash recovery. The log is the only structure on disk; it contains indexing information so that files can be read back from the log efficiently. In order to maintain large free areas on disk for fast writing, we divide the log into *segments* and use a *segment cleaner* to compress the live information from heavily fragmented segments. We present a series of simulations that demonstrate the efficiency of a simple cleaning policy based on cost and benefit. We have implemented a prototype log-structured file system called Sprite LFS; it outperforms current Unix file systems by an order of magnitude for small-file writes while matching or exceeding Unix performance for reads and large writes. Even when the overhead for cleaning is included, Sprite LFS can use 70% of the disk bandwidth for writing, whereas Unix file systems typically can use only 5–10%.

Categories and Subject Descriptors: D 4.2 [**Operating Systems**]: Storage Management—*allocation/deallocation strategies; secondary storage*; D 4.3 [**Operating Systems**]: File Systems Management—*file organization, directory structures, access methods*; D.4.5 [**Operating Systems**]: Reliability—*checkpoint/restart*; D.4.8 [**Operating Systems**]: Performance—*measurements, simulation, operation analysis*; H.2.2 [**Database Management**]: Physical Design—*recovery and restart*; H.3 2 [**Information Systems**]: Information Storage—*file organization*

General Terms: Algorithms, Design, Measurement, Performance

Additional Key Words and Phrases: Disk storage management, fast crash recovery, file system organization, file system performance, high write performance, logging, log-structured, Unix

A Log-structured file system (**SpriteFS**) was investigated back in the early 1990s

Highly influential work

Can you guess why such a design would make sense back in 1992 for a HDD based fs?

Mendel Rosenblum and John K. Ousterhout. **1992**. The design and implementation of a log-structured file system. *ACM Trans. Comput. Syst.* 10, 1 (Feb. 1992), 26–52. DOI:https://doi.org/10.1145/146941.146943

Why Log-Structured file system (LFS) in 1992?

1. The amount of system DRAM was increasing
    a. More opportunity to cache data and serve “read” requests from DRAM
    b. DRAM is random access, hence, good “read” performance

2. Access to disk will dominated by “writes”
    a. Writes can be sequential and random
    b. Writes can be **small** and **large** - large writes are OK, *but small writes are really bad. Plus random writes for metadata updates --- really really bad*

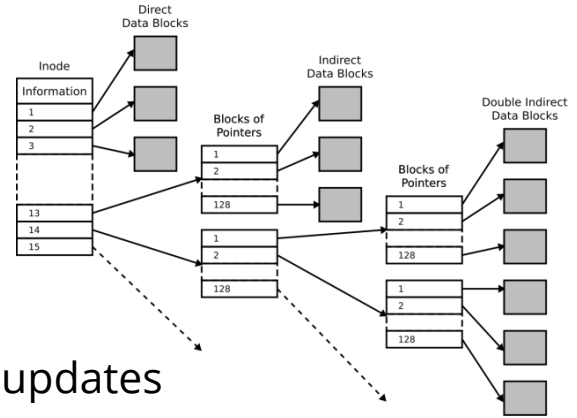
3. Hence, use a log-structured file system optimized for servicing fast writes

It turned out that “log” is a very useful data structure for write-once media as well (like NAND flash). *But how do you make a working file system on a log? How to do you find inodes? and what happens when a log is full?*

The basic idea of an LFS

With an LFS, there cannot be a single known location where inodes are stored, the location changes every time an inode is updated

LFS's goal is to optimize inode metadata lookups -- *why*?



All new writes are written to the log in a sequential manner, and then a "`inode map`" structure is written to identify their locations

inode maps are written to the log after each (or batch of) updates

LFS's checkpoint region contains all inode maps information

Inode maps are typically cached in the buffer cache for fast lookups

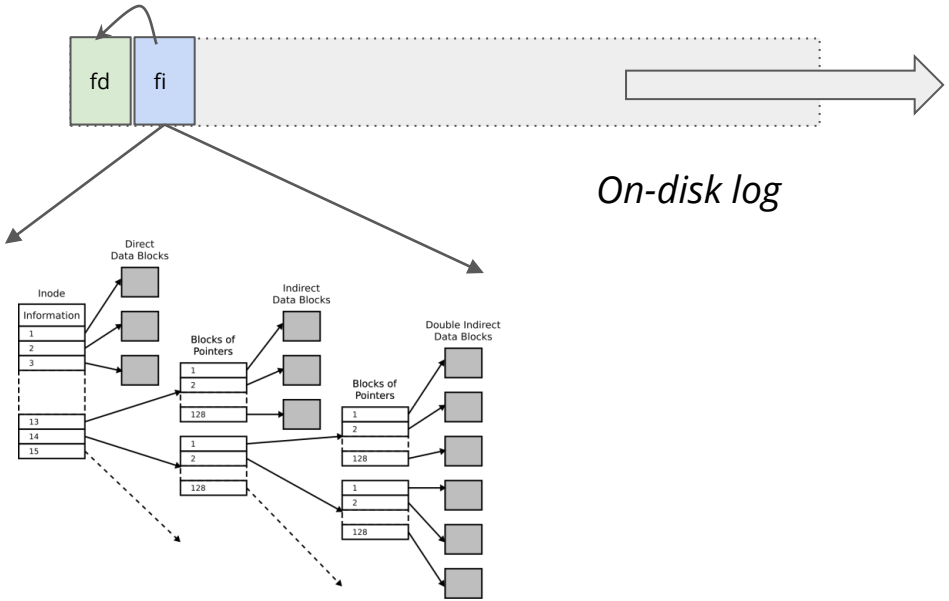
http://www.cs.cornell.edu/courses/cs4410/2014fa/slides/13-lfs.pdf

Simple example

Let’s say we want to create `/dir1/file1`

Data block

File inode block



File inode contains the disk offsets for these pointers

Simple example

Let's say we want to create `/dir1/file1`

Data block

File inode block

Directory data block

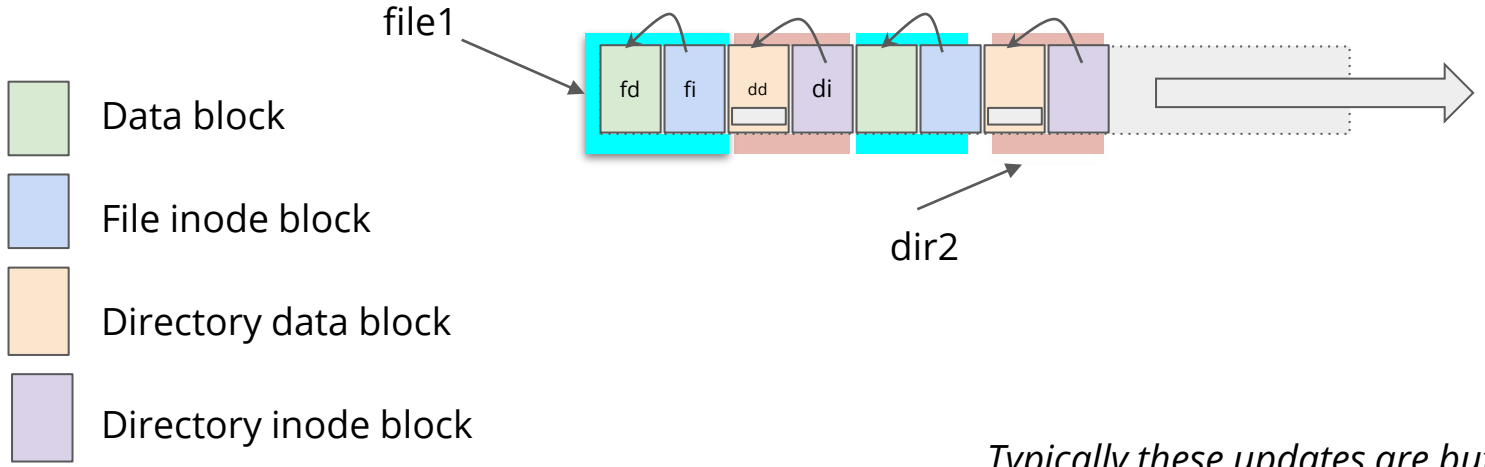
Directory inode block

fd = file data
fi = file inode
dd = directory data
di = directory inode

Remember directories are just special files with a special format to keep track of all other files and directories inside it

Simple example

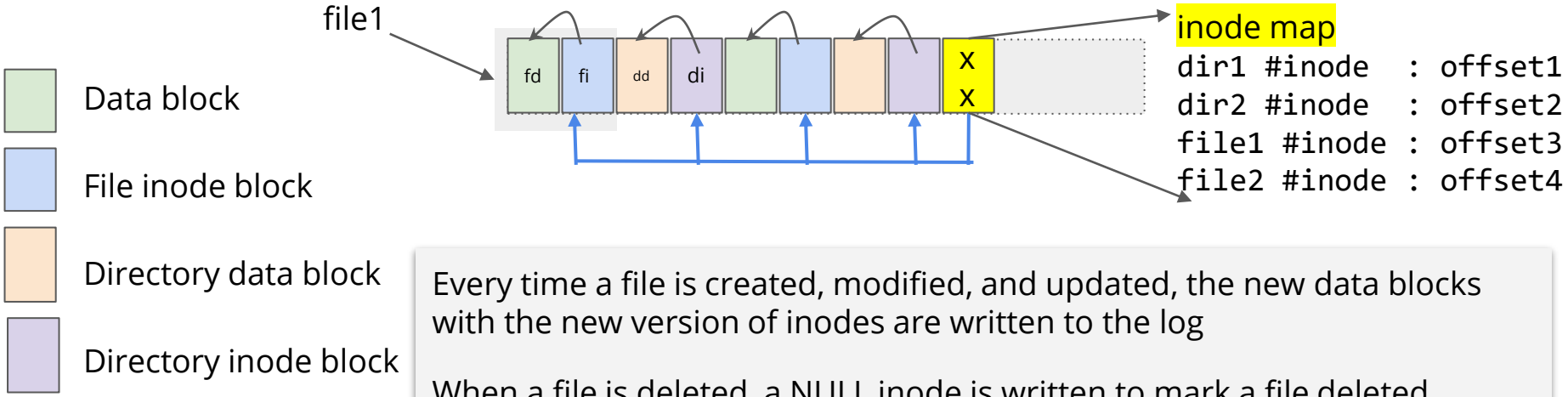
Let’s say we want to create another `/dir2/file2`



*Typically these updates are buffered in the DRAM cache and then written out in **a single large sequential segments** to amortize the disk seek cost*

Simple example

Two locations: `/dir1/file1` and `/dir2/file2`



Every time a file is created, modified, and updated, the new data blocks with the new version of inodes are written to the log

When a file is deleted, a NULL inode is written to mark a file deleted

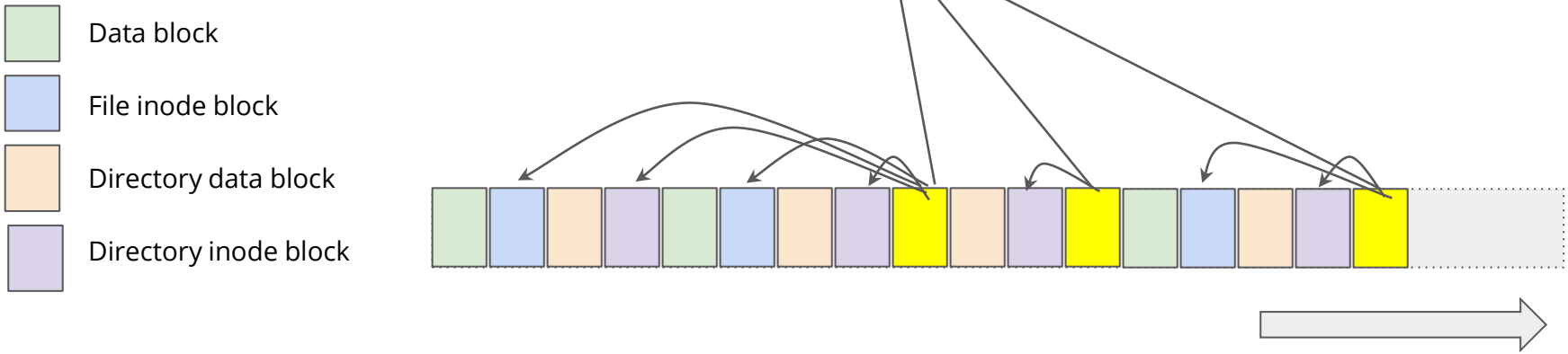
There is **an in-memory big inode map table** that has the latest offset entries for all inode locations written to the log (*optimization, not necessary for the correctness*)

Simple example

In-memory Inode map Table

```
Inode number 100 : offset1
Inode number 200 : offset2
...
```

In case there are concurrent updates
→ The last one wins
→ in case there is a crash, the in-memory table can be build again by scanning the log at the mount time

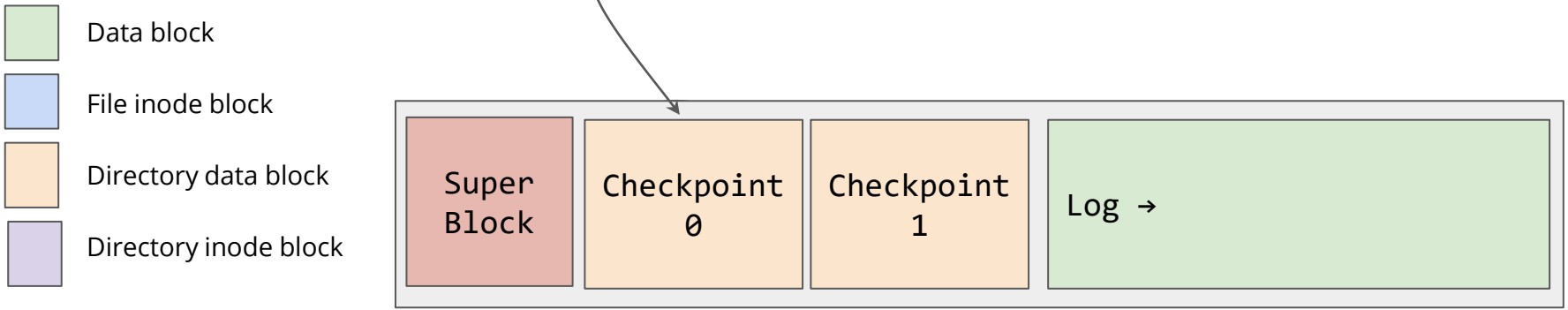


Simple example

In-memory Inode map Table

```
Inode number 100 : offset1
Inode number 200 : offset2
...
```

In case there are concurrent updates
→ The last one wins
→ in case there is a crash, the in-memory table can be build again by scanning the log at the mount time

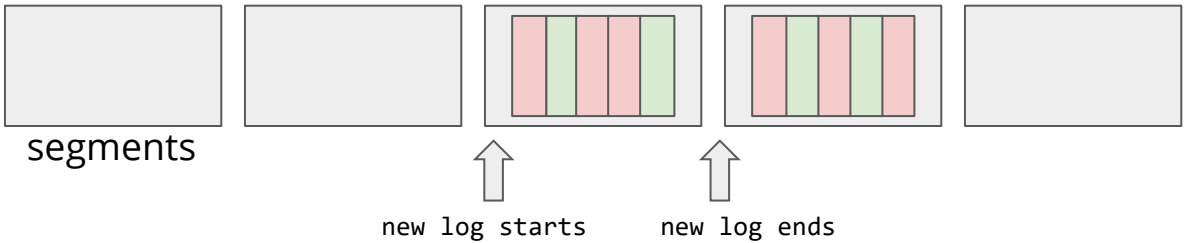


The one thing it has to remember is where is **the root inode location** - that can be stored when the LogFS does checkpointing (like any other file system). The initial SuperBlock location and 2x **checkpoint regions are fixed** (stores inode map table, root inode location etc.)

What happens when a log become full?

The log is divided into large “segments” which are the unit of cleaning (typically 10-100MBs, anyone Zones?)

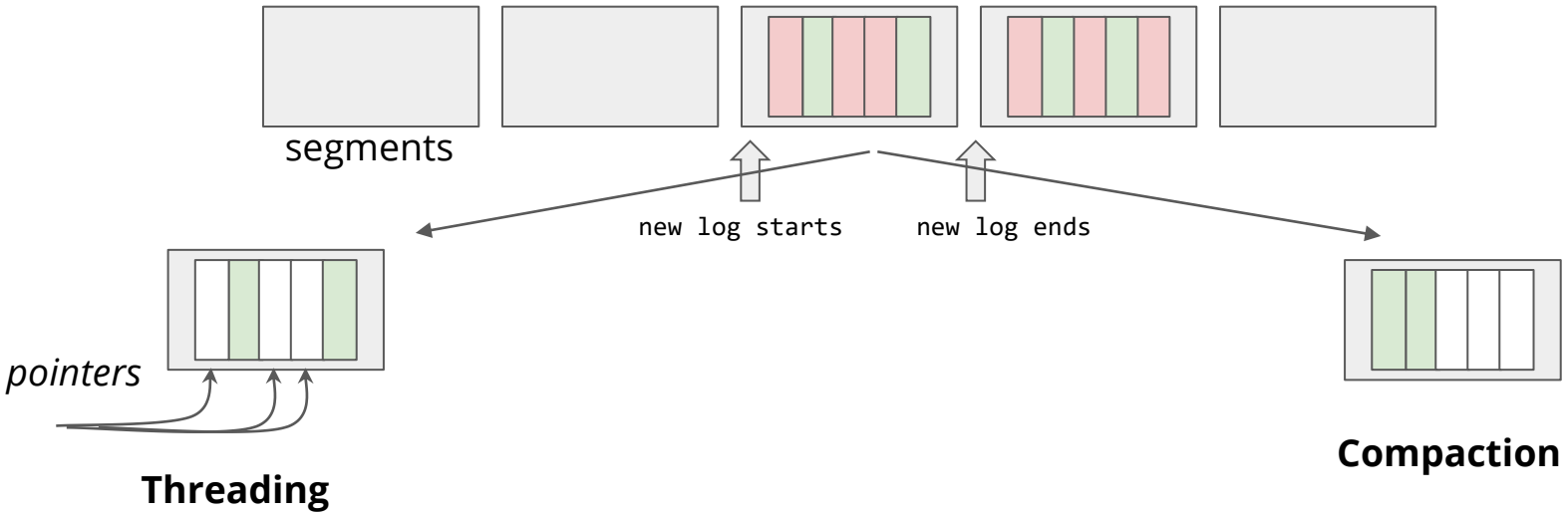
So what can be done with this design? **Two options:**



What happens when a log become full?

The log is divided into large “segments” which are the unit of cleaning (typically 10-100MBs, anyone Zones?)

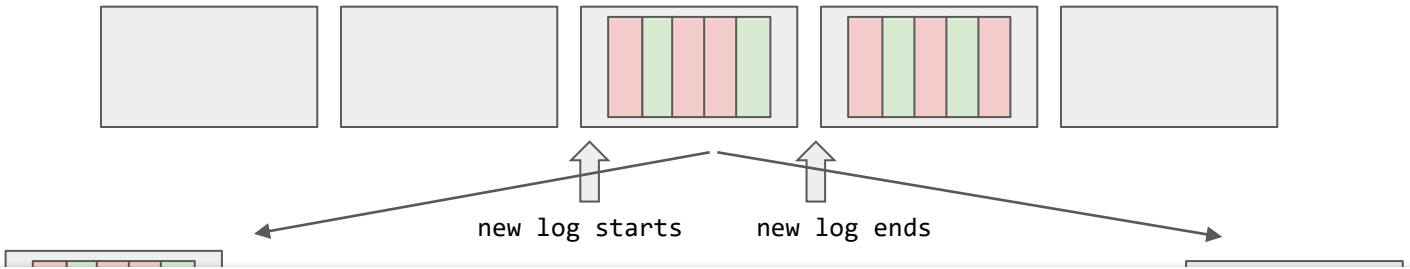
So what can be done with this design? **Two options:**



What happens when a log become full?

The log is divided into large “segments” which are the unit of cleaning (typically 10-100MBs, anyone Zones?)

So what can be done with this design? **Two options:**



Threading : no explicit garbage collection, fast, but metadata to keep track of holes, random accesses

Compaction : explicit garbage collection phase, copy cost, but gives nice clean blocks with less overheads

Sprite LFS used a hybrid: segments is always written sequentially and then copy and compacted
However, the log is threaded segment-by-segment basis

The log cleaning

Segments are the unit of GC cleaning

After each segment there is a segment summary block to keep track of “live” and “dead” blocks

Everytime GC is invoked - it need to select a target/victim segment for cleaning

At the time of cleaning, when data is being re-arranged, the GC has an opportunity to re-arrange blocks in a segment to pack “hot and cold” data separately

Segment cleaning logic: Picking up a victim

Greedy

$$\text{write cost} = \frac{\text{total bytes read and written}}{\text{new data written}} = \frac{\text{read segs} + \text{write live} + \text{write new}}{\text{new data written}} = \frac{N + N*u + N*(1-u)}{N*(1-u)} = \frac{2}{1-u}$$

Cost-Benefit Analysis

$$\frac{\text{benefit}}{\text{cost}} = \frac{\text{free space generated}*\text{age of data}}{\text{cost}} = \frac{(1-u)*\text{age}}{1+u}.$$

Goal: the goal is to create one clean segment for every new segment data written
Greedy picks up the most utilized segment ("u" is utilization between 0 and 1), "N" is the number of pages in a segment

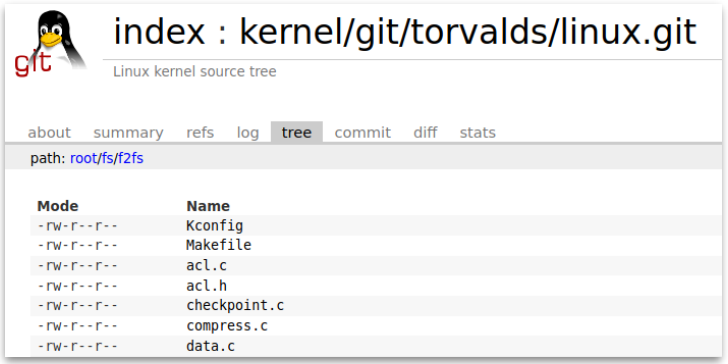
Cost-benefit analysis does include the "hotness" or "age" of data (the last time data was updated) and how much space we will free (1 - u), with total work (1 (read) + u (write))

Why Log-Structured FSes were not good enough

1. **Segment cleaning overheads in Log-Structured file systems**
    a. The Achilles heel for LogFS : ***a long and interesting debate***
        i. *"the impact of the cleaner is so severe that BSD-LFS cannot compete with either FFS or EFS. For the tests presented here, the disk was running at 85% utilization, and the cleaner was continually running. Note that FFS and EFS allocate up to 90% of the disk capacity without exhibiting any performance degradation" (https://www.hhhh.org/perseant/lfs/lfs_for_unix.pdf, page 16)*
    b. How would you identify hot/cold data? Is there a FS API?

2. **Ignoring the device characteristics**
    a. Different sector, page, block sizes and layouts
    b. Multiple read/write possible at the same time
    c. Not all random writes are the same
    d. Different read, write, and GC cost and granularities
    e. Performance vs. utilization

Flash-Friendly File System (F2FS) (2015)

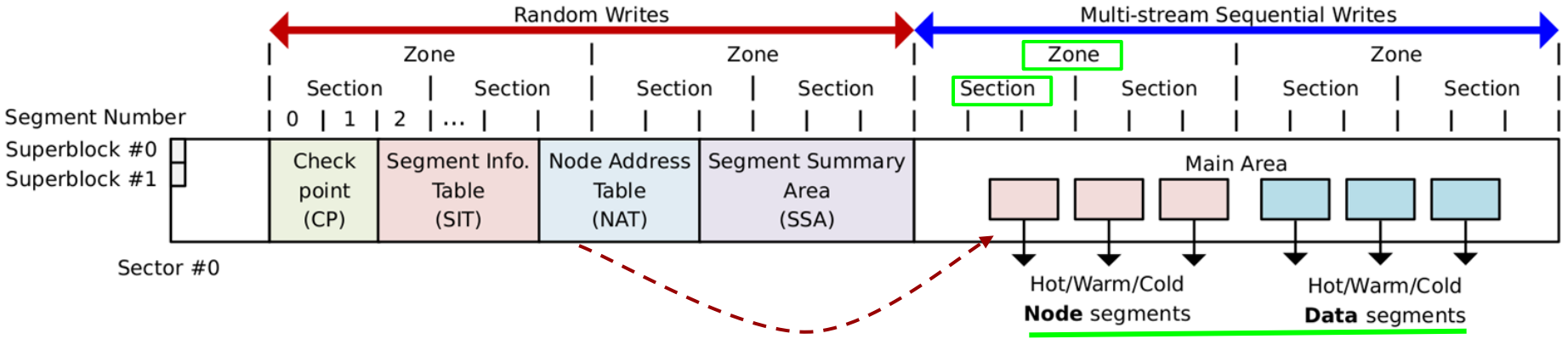


Highly influential work

One of the first file systems designed from scratch for NAND flash and is part of the mainline kernel (**production quality**):
https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/fs/f2fs?h=master
https://www.kernel.org/doc/html/latest/filesystems/f2fs.html

Primary concerns: *layout and parallelism inside flash devices* (+previous best ideas)

F2FS: Disk Layout



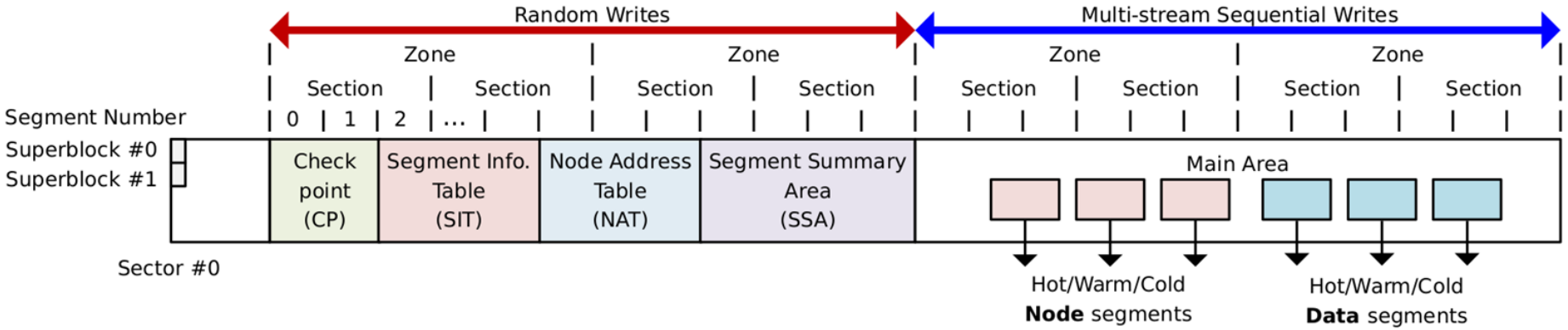
Device is split into:

- Zones : unit of parallelism, can open multiple parallel zones for I/O
  - Sections **: unit of cleaning, some multiple of the flash GC units** (if known, or large enough)
    - Segments : unit of space allocation (can contain multiple flash pages)

A section stores either (1) **Node** contains inode (with single, double, triple pointer pages) and indices of data pages; or (2) **Data** segments (user data)

Two areas: random writes (F2FS's own metadata) and sequential writes

F2FS: Disk Layout

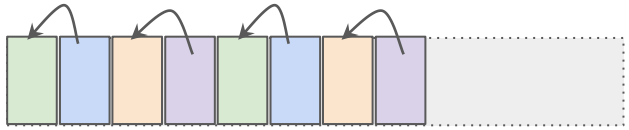


All the file metadata is written in the start Zones (classified as **Random Write Zones**)

- Superblock : read-only information about the file system
- Check point area (CP) : 2x to switch between stable and active
- Segment Information Table (SIT) : per-segment information, live blocks, used in GC
- **Node Address Table (NAT) : address of "nodes" blocks in the Main Area**
- Segment Summary Area (SSA) : to identify parent blocks and fs tree
- **Main Area : data (metadata and data) segments are written**

F2FS : File Structure

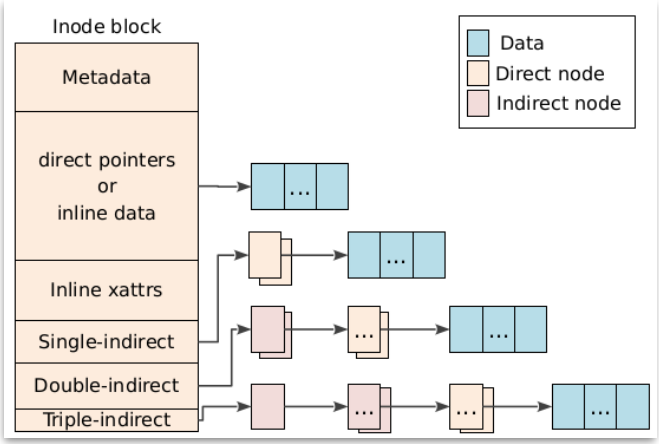
The file structure is not surprising, follows a typical “inode” based tree model



There are direct, single, double, and triple indirect pointers

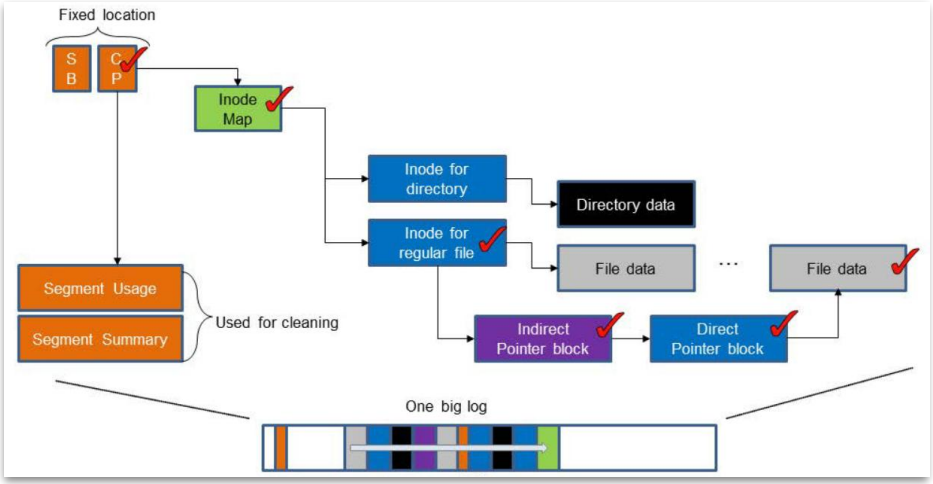
In original LFS: there is an **inode map** to translate an inode number to an on-device location (written at the end of the segment)

F2FS uses **the NAT table** to translate an inode number to its on-device location



This design solves an important problem with log-based file system: **Recursive update problem**

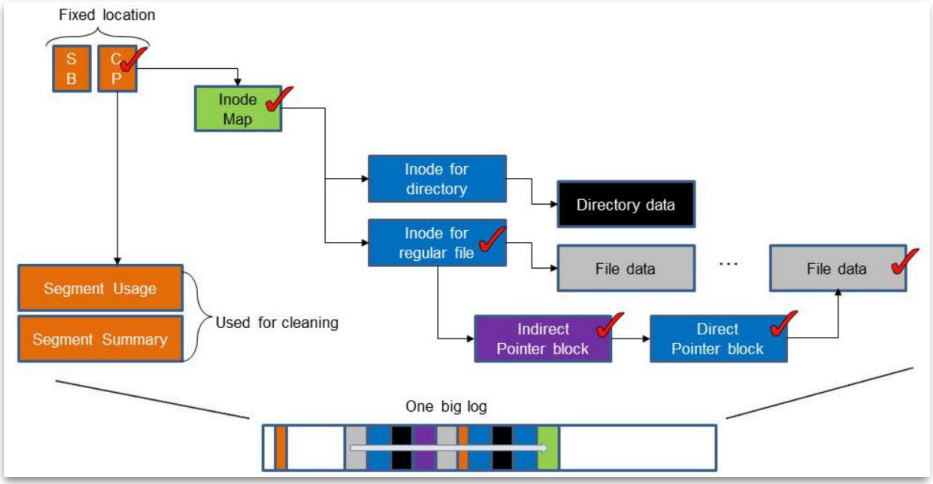
Update Propagation in LFS vs. F2FS

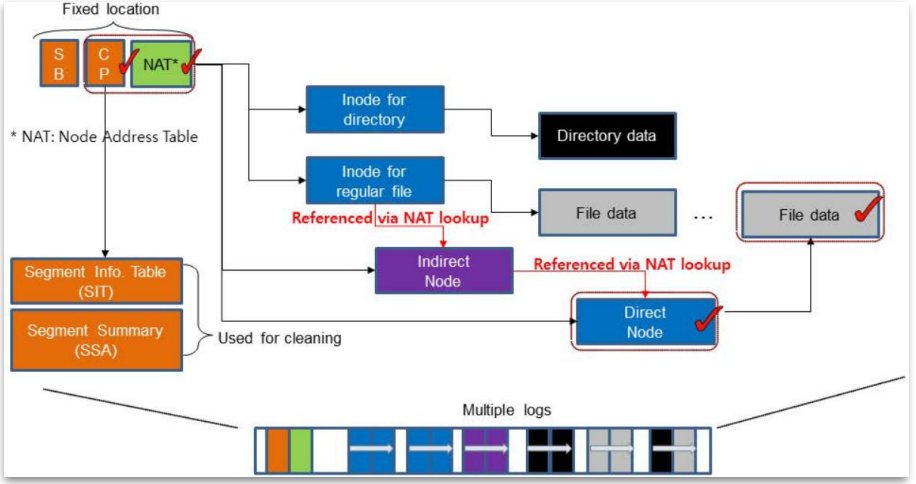


In a Log-Structured file system, updates at the bottom of the tree will be bubbled through the whole tree (updating device addresses) until reached at the top and a new inode map location is written - this is called ***recursive update problem*** *(also known as* ***Wandering Tree problem****)*

https://www.usenix.org/sites/default/files/conference/protected-files/fast15_slides_lee.pdf

Update Propagation in LFS vs. F2FS





In a Log-Structured file system, updates at the bottom of the tree will be bubbled through the whole tree (updating device addresses) until reached at the top and a new inode map location is written - this is called ***recursive update problem*** *(also known as* ***Wandering Tree problem****)*

In contrast, F2FS uses node numbers for indexing and only immediate parent is updated with further updates in-place in NAT (which is at a fixed location, Node Number → Device Address)

https://www.usenix.org/sites/default/files/conference/protected-files/fast15_slides_lee.pdf

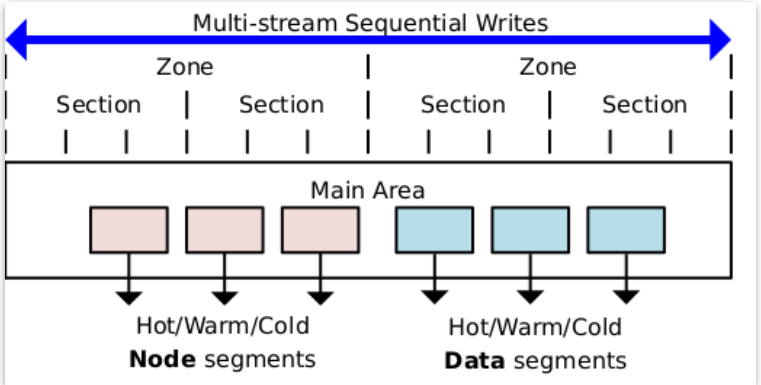
Multi-Headed stream logging

F2FS leverages device parallelism by opening multiple write segment streams

These streams are classified based on their hotness and separated in zones

- Uses a simple classification (unlike SFS)
- Different types (table) put in different classes

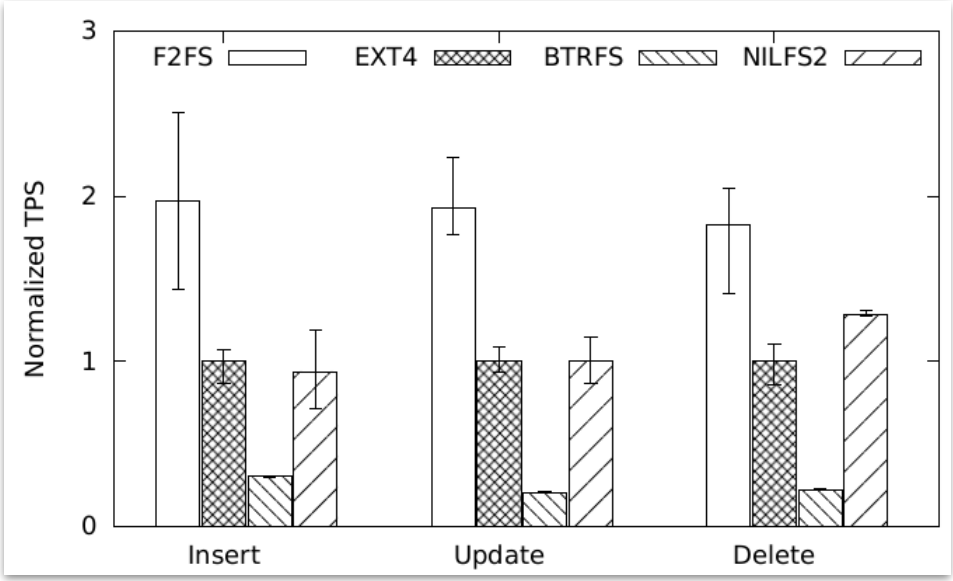
Different zones are mapped to different parallel units inside the flash



Type	Temp.	Objects
Node	Hot	Direct node blocks for directories
	Warm	Direct node blocks for regular files
	Cold	Indirect node blocks
Data	Hot	Directory entry blocks
	Warm	Data blocks made by users
	Cold	Data blocks moved by cleaning; Cold data blocks specified by users; Multimedia file data

F2FS performance

Thesis topic: *these numbers ought to be updated for the fastest flash we have*



SQLite workload on 3 different file systems, F2FS outperforms them all

(more detailed performance evaluation in the paper)

F2FS recap

Key choices in the design of F2FS

1. Flash-friendly data layouts : align fs GC unit (segments) with FTL gc unit
2. NAT updates to restrain writes update propagations
    a. Accepts random writes for the FS metadata regions
3. Multi-headed logging for parallelism

So far

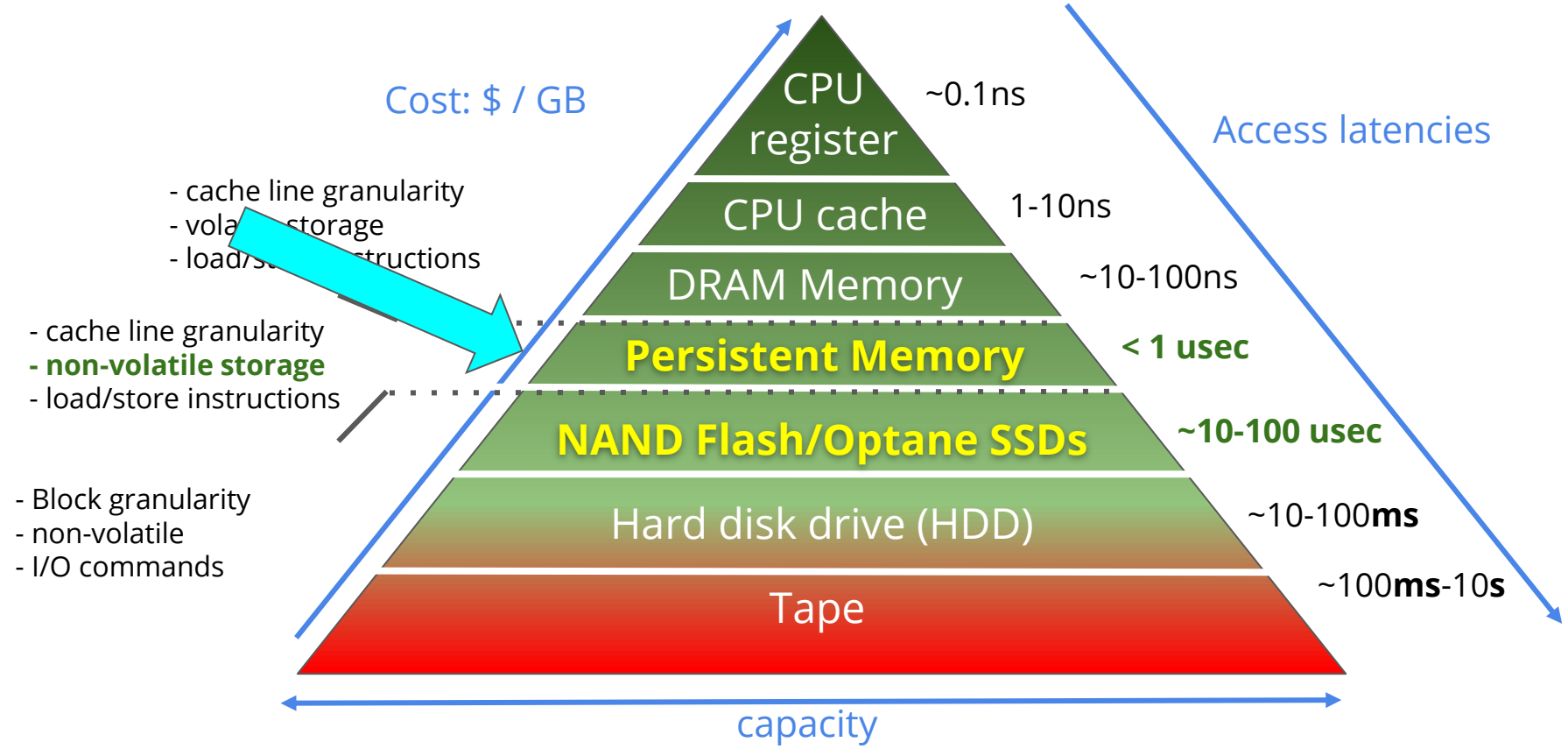
You have seen the original **Log-Structured File System design** (Sprite FS)

- Design originally for disks, but fits perfectly with NAND flash too :)
- Typically “GC” is the Achilles Heel of any log-structured file system

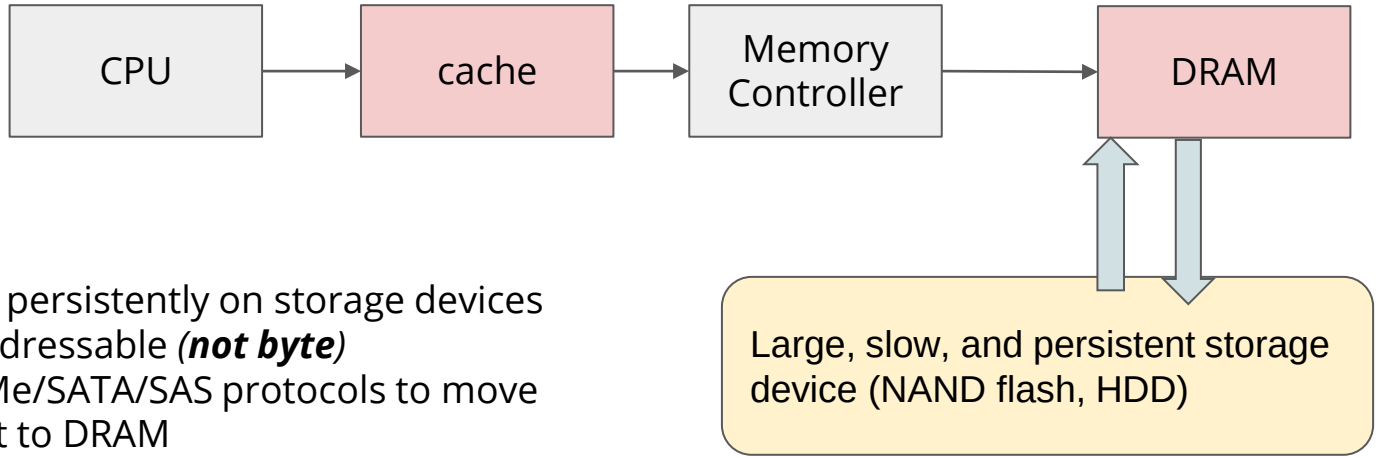
F2FS, flash-friendly layouts with with *multi-headed* logging capabilities

NVM (Persistent Memory) for File System

The (new) triangle of storage hierarchy



The Basic Storage Model

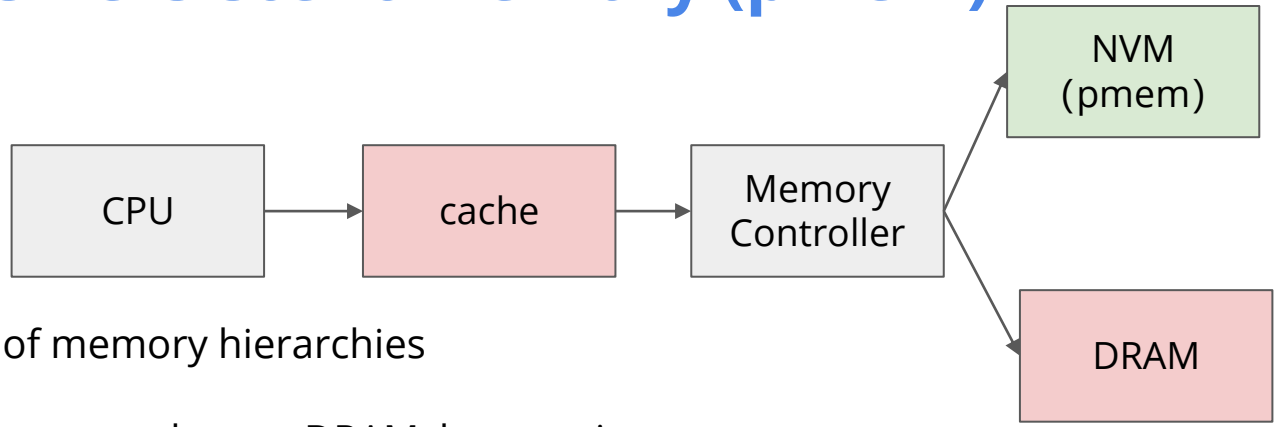


Data is stored persistently on storage devices

- Block addressable *(**not byte**)*
- Use NVMe/SATA/SAS protocols to move data first to DRAM
- CPU can _*only*_ access data from DRAM
- To make data persistent write out again to the storage - responsibility of the application/OSes

The two-level of storage hierarchy: **Memory** (fast, byte-addressable, small, volatile) and **Storage** (slower, block-addressed, large, and non-volatile)

NVM as Persistent Memory (pmem)



The holy grail of memory hierarchies

Ideally: performance close to DRAM, but persistent

We have been anticipating these memories from many years, and hence, continued to do research in the “software” architectures before they arrived
→ *In this lecture we will cover the high-level concepts (it is a large area, see references)*

Keep in mind that **the abstraction of persistent memories** is much border and can be done on conventional devices also with mmap: https://web.eecs.umich.edu/~tpkelly/papers/Failure_atomic_msync_EuroSys_2013.pdf and https://www.usenix.org/system/files/login/articles/login_winter19_08_kelly.pdf

Today: Intel Optane

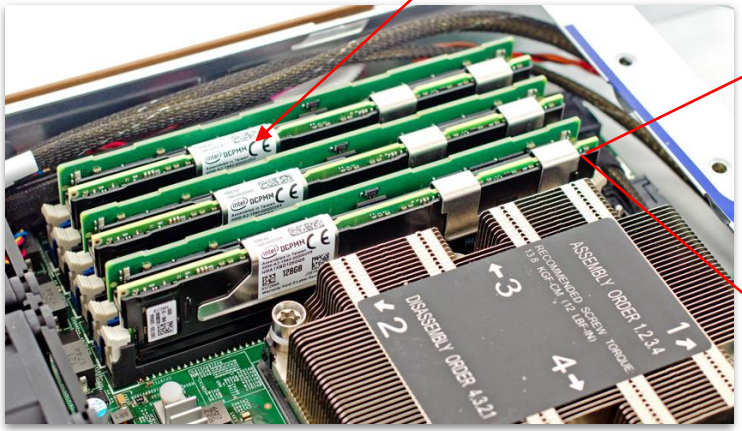
Released in **2019** (latest and greatest piece of storage technology ~~today~~)

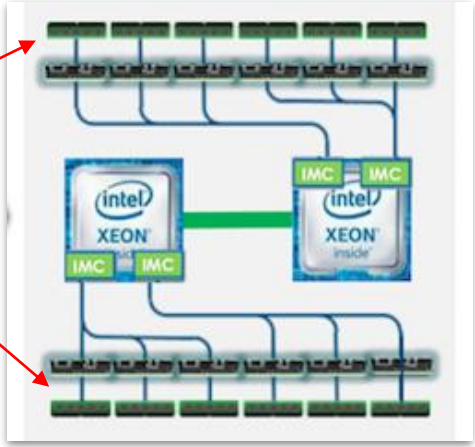
It is a byte-addressable, load-store accessible (from the CPU) storage that can be put in a DDR4 DIMM slot (uses the same mechanical and electrical protocols)

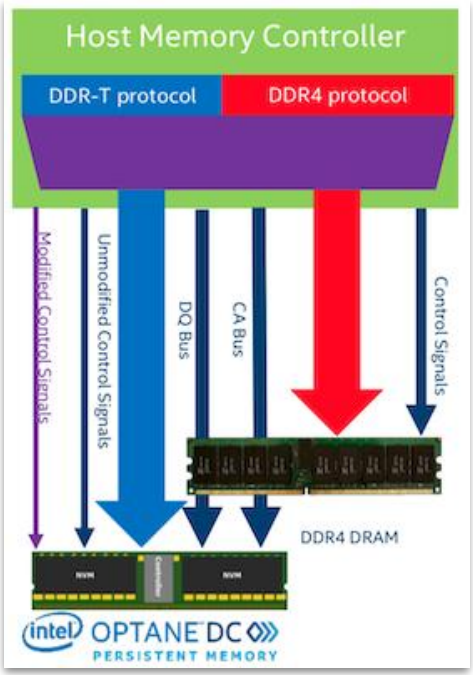
In comparison to DRAM

- **More capacity :** 128, 256, and 512 GB DIMMs (DRAMs are usually at 32-64GB then they get super expensive)
- **Cheaper :** than the DRAM (2-4x) times, but more expensive than Flash (10-100x)
- **Energy Efficient :** Unlike DRAM, no need to constantly refresh

Today: Intel Optane

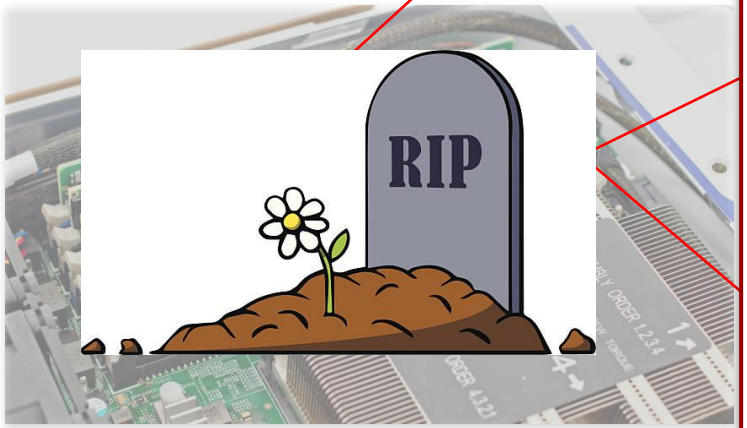






Intel Optane DC Persistent Memory Module (PMM), https://www.storagereview.com/news/intel-optane-dc-persistent-memory-module-pmm

~~Today: Intel Optane~~



Intel Kills Optane Memory Business, Pays $559 Million Inventory Write-Off

By Paul Alcorn published July 28, 2022

3D XPoint at the last crossroad.

Comments (31)

(Image credit: Lenovo)

Update 08/02/2022 12:30am PT: Intel reached out to clarify that it would bring the next-gen Crow Pass Optane memory DIMMs to market and will use its existing inventory to fulfill orders. This wasn't clear from Intel's previous statement because this is technically a future product. We have clarified that point in the below text.

tane Memory, writes
inventory
hip giant

business, a line of memory that was slightly slower than
gh input/output operations per second.

59 million inventory impairment/write-off as it exits the

in a
ing
ctives.
ease
ane

– Intel

ory chip business to SK Hynix, but kept onto Optane - a
nology it developed with Micron in 2015.

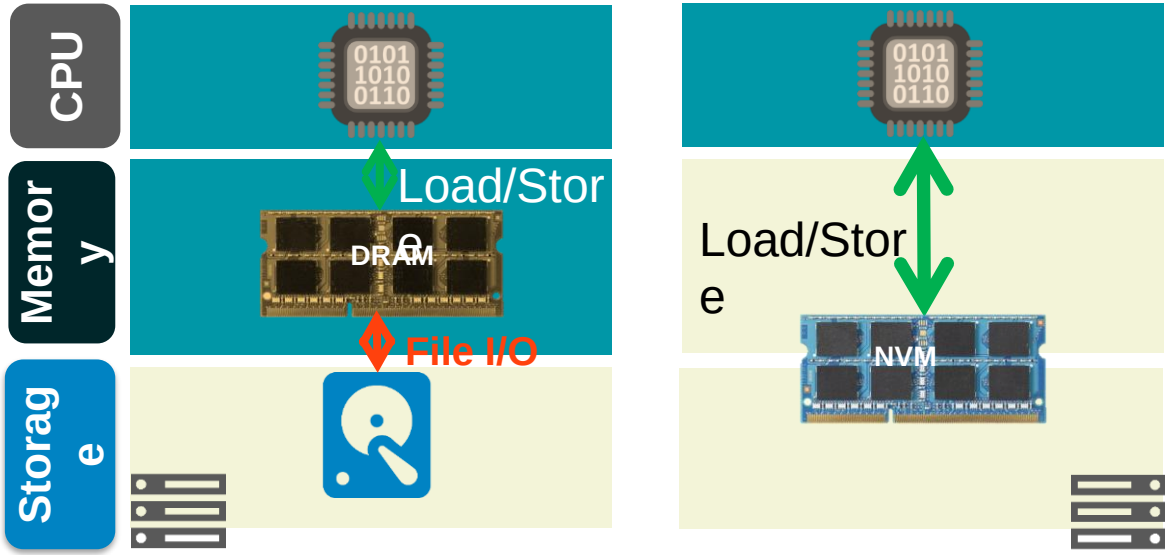
persistent memory DIMMs based on the technology,
market, but was a complete failure in the consumer
on in January 2021.

ted in 2021, and left the market.

https://www.tomshardware.com/news/intel-kills-optane-memory-business-for-good

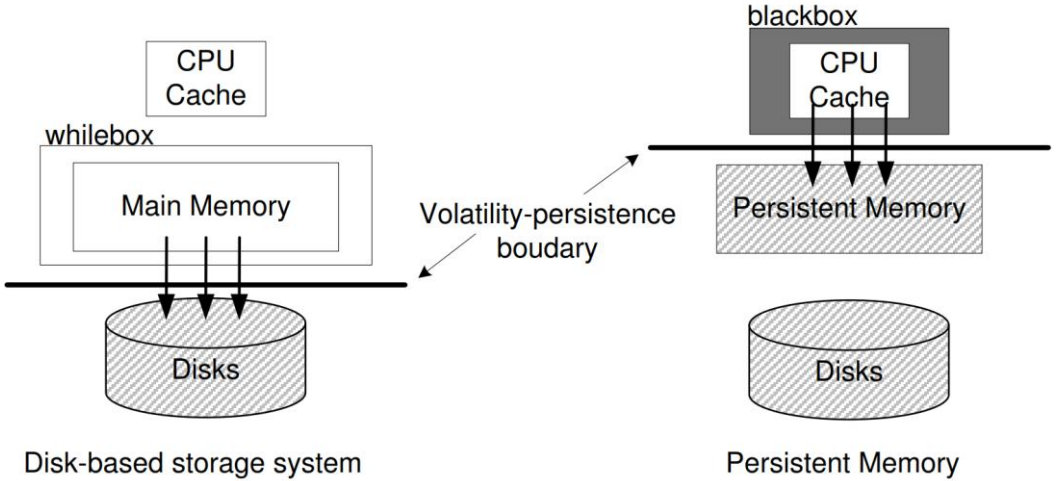
NVM brings huge changes

- **NVM changes the memory hierarchy**
  - Memory + Storage ➔ Persistent Memory
  - From **two layer** to **single layer**
  - Interfaces changed: Block I/O → Load/Store



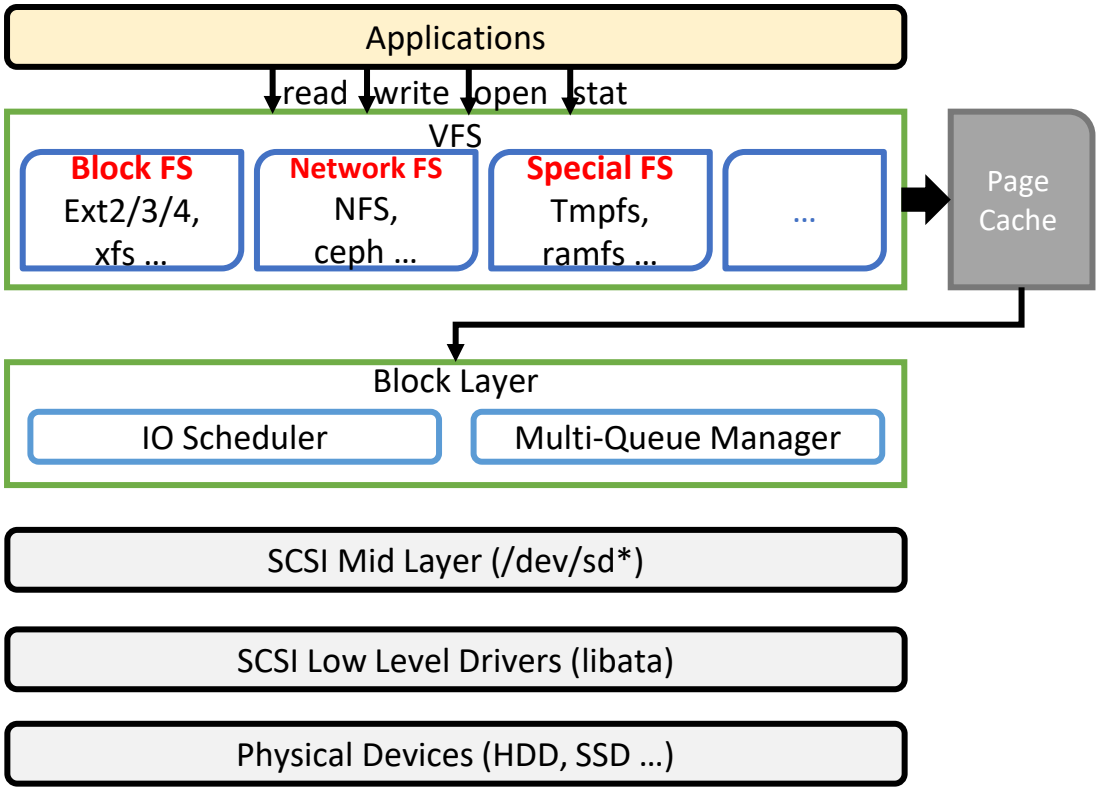
NVM brings huge changes

- **NVM changes the volatility-persistence boundary**
  - Between main memory and disks
  - Data movements are software-controlled (**whitebox**)
  - Between CPU cache and NVM
  - hardware controlled (**blackbox,** out-of-order execution)



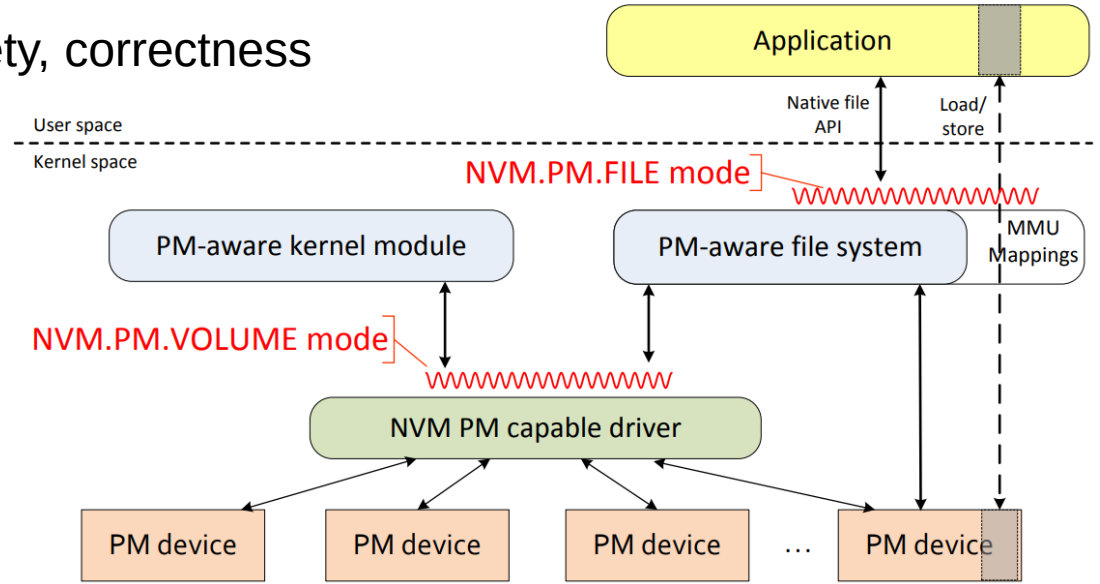
Build NVM-aware storage systems is hard!

- I. How to abstract the NVM device?



Build NVM-aware storage systems is hard!

- **I. How to abstract the NVM device?**
  - NVM can be accessed with Load/Store instruction
  - File, object, or just a naïve heap?
  - Kernel or user level?
  - Safety, correctness



Source: https://www.snia.org/sites/default/files/NVMProgrammingModel_v1.pdf

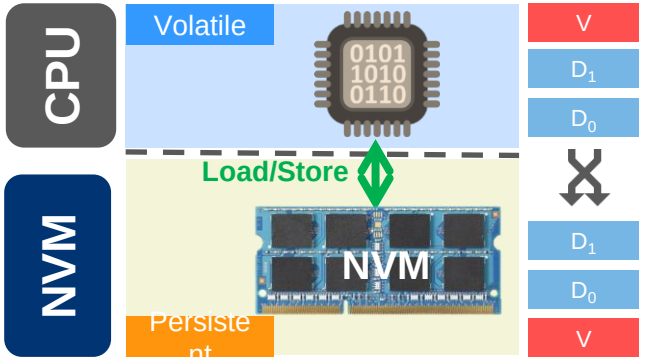
Build NVM-aware storage systems is hard!

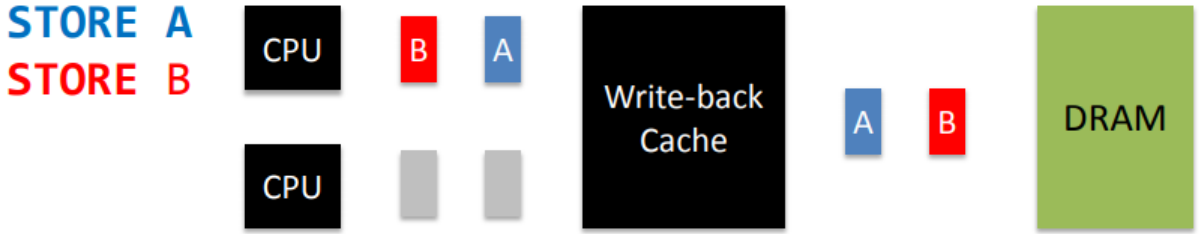
- **II. How to guarantee the crash consistency?**

Data should keep consistent after a system/power failure

- can be read again after reboot
- ❑ CPU cache (e.g., write-back) reorders writes to NVM, preventing us to correctly recover the data

```
STORE data[0] = 0xFOOD
STORE data[1] = 0xBEEF
STORE valid = 1
```





Build NVM-aware storage systems is hard!

- II. How to guarantee the crash consistency?

Volatile memory

```
if (valid) {
    Load(data[0]);
    Load(data[1]);
}
```

```
1: STORE data[0] = 0xFOOD
2: STORE data[1] = 0xBEEF
3: STORE valid = 1
```

Build NVM-aware storage systems is hard!

- **II. How to guarantee the crash consistency?**

Volatile memory

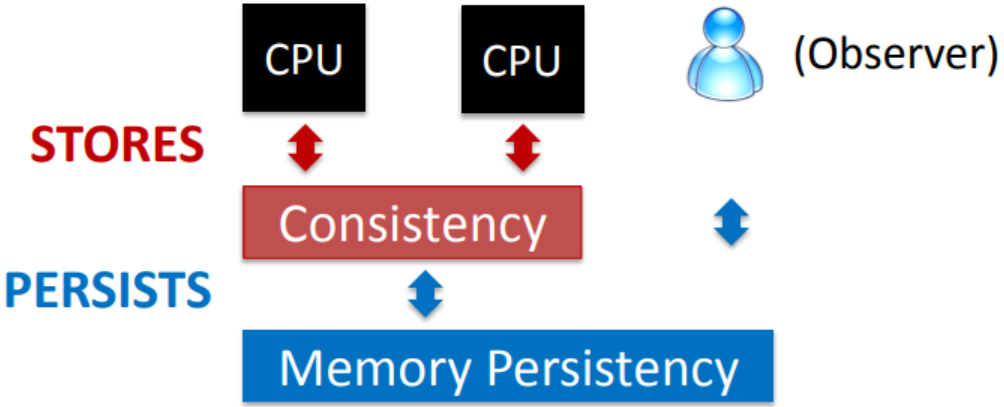
Reordering to DRAM does not break correctness

Memory consistency orders stores between CPUs

Persistent memory

Build NVM-aware storage systems is hard!

- **II. How to guarantee the crash consistency?**



- Memory consistency
  - Constrains order of loads and stores between CPUs
- Memory persistency
  - Constrains order of writes with respect to failure

Build NVM-aware storage systems is hard!

- **II. How to guarantee the crash consistency?**
  - Simple solutions
    - Disable caching
    - Write-through caching
    - Flush entire cache at commit

Build NVM-aware storage systems is hard!

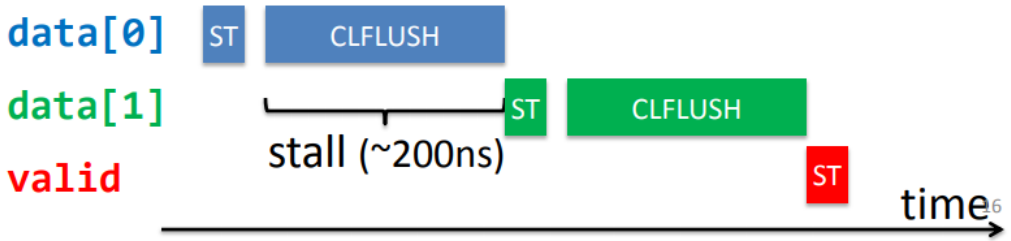
- **II. How to guarantee the crash consistency?**

Order writes by flushing cachelines via CLFLUSH

```
STORE data[0] = 0xFOOD
STORE data[1] = 0xBEEF
CLFLUSH data[0]
CLFLUSH data[1]
STORE valid = 1
```

But CLFLUSH:

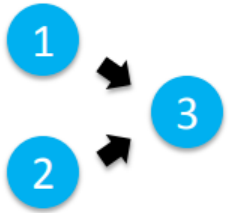
– Stalls the CPU pipeline and serializes execution



Build NVM-aware storage systems is hard!

- **II. How to guarantee the crash consistency?**

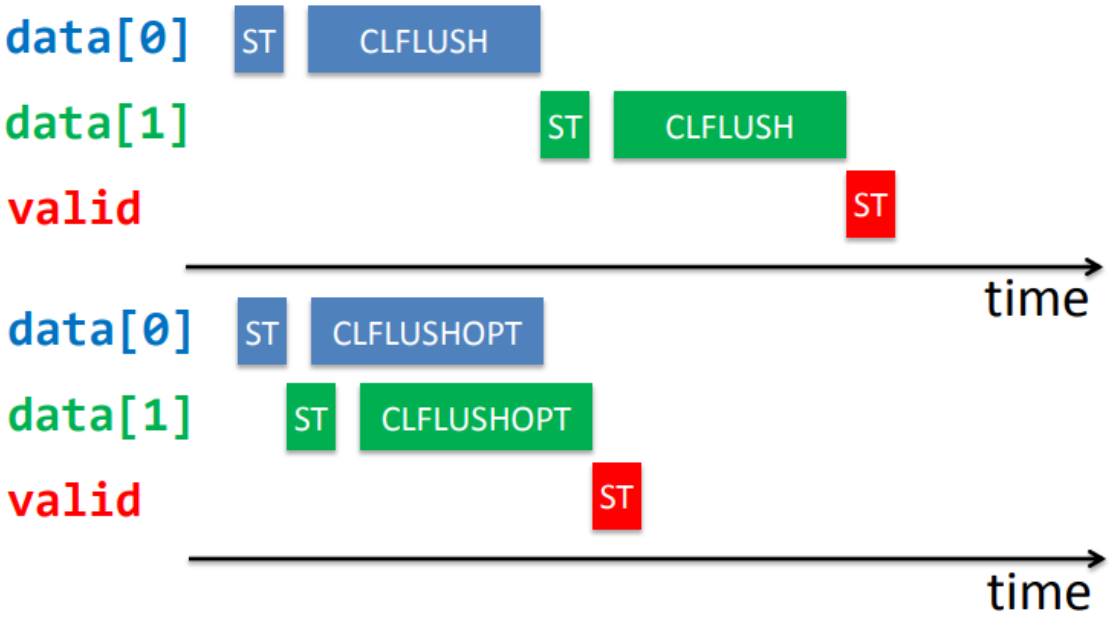
```
1: PERSIST data[0]
2: PERSIST data[1]
3: PERSIST valid
```



Need interface to expose necessary constraints

Build NVM-aware storage systems is hard!

- II. How to guarantee the crash consistency?
  - CLFLUSHOPT



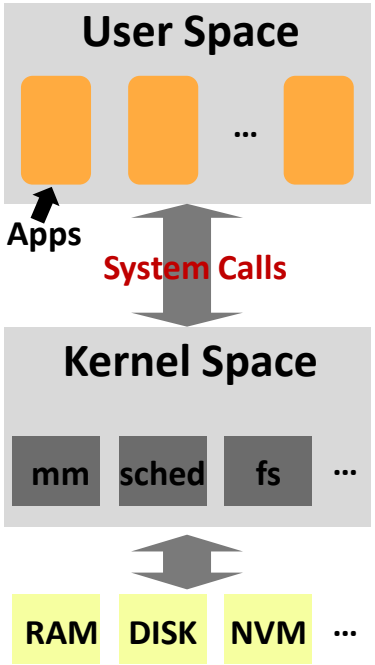
Build NVM-aware storage systems is hard!

- **II. How to guarantee the crash consistency?**
  - Barrier
    - mfence, sfence
    - Following cacheline flush instructions
  - Example

```
clflushopt data[0];
clflushopt data[1];
mfence();
valid = 1;
clflushopt valid
```

NVM-aware File System

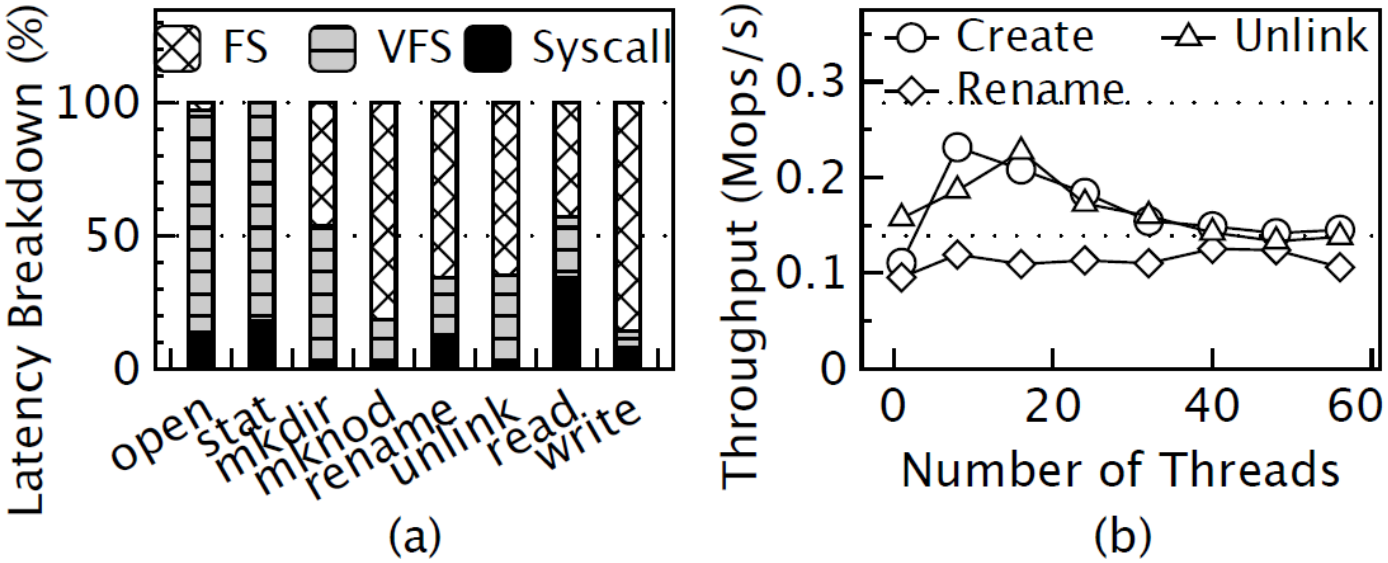
- **Inefficiency of existing OS abstraction**
  - System calls
    - Save/restore context
    - CPU cache eviction
    - TLB pollution
  - Virtual File System (VFS)
    - Redundant DRAM cache (page/metadata cache)
    - Coarse-grained lock managements
    - Heavy-weight software stack



OS-part overhead is magnified when the storage devices are getting faster.

NVM-aware File System

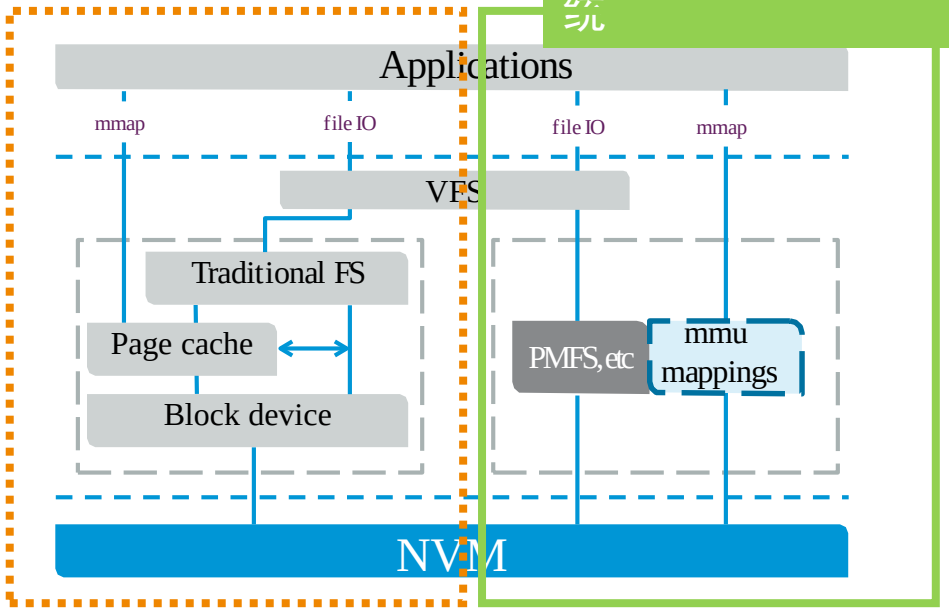
- **Inefficiency of existing OS abstraction**



NVM-aware File System

通过模拟块设备的形式兼容传统文件系统

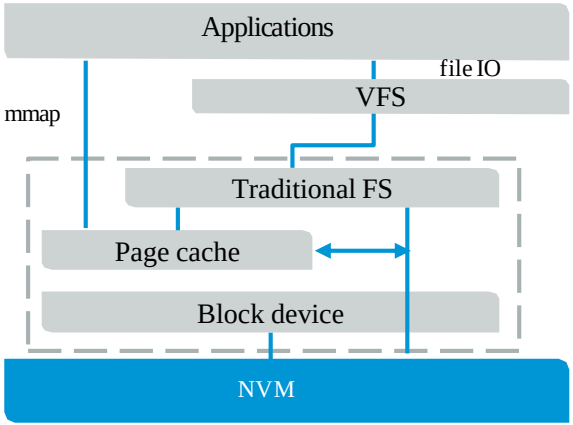
基于持久性内存重新构建字节粒度的持久性文件系统



NVM-aware File System

一、通过RAMDISK模拟块设备兼容传统文件系统

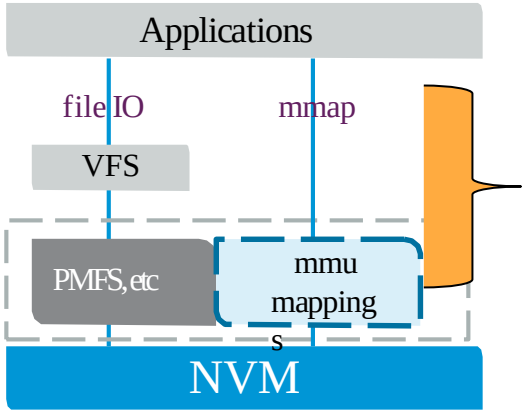
- 传统文件系统无需修改，可直接构建于以持久性内存模拟的RAMDISK块设备之上，如 EXT4，BtrFS
- RAMDISK形式使得传统文件系统快速受益于内存级的数据持久化，相比于外存性能有数量级的提升
- 不足：软件层的开销巨大，无法充分利用持久性存储介质优势



NVM-aware File System

基于持久性内存重新构建字节粒度文件系统

- 细粒度的数据访问
- 融合内外存管理方式
- NVM直写，避免双重拷贝和块层开销



代表性工作：

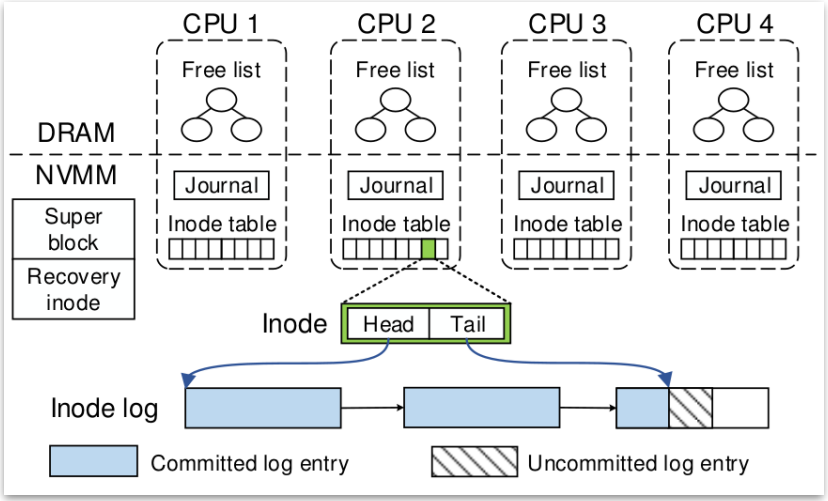
1. 轻量级持久化内存文件系统PMFS [EuroSys'14]
2. 日志结构持久性文件系统NOVA [FAST'16]
3. 在传统存储栈中利用NVM的NVLog [FAST'25]

NOVA Design and Ideas

A Log-structured file system

Each inode has its own log (concurrency and parallelism)

Each CPU has its own set of inodes and free list to manage

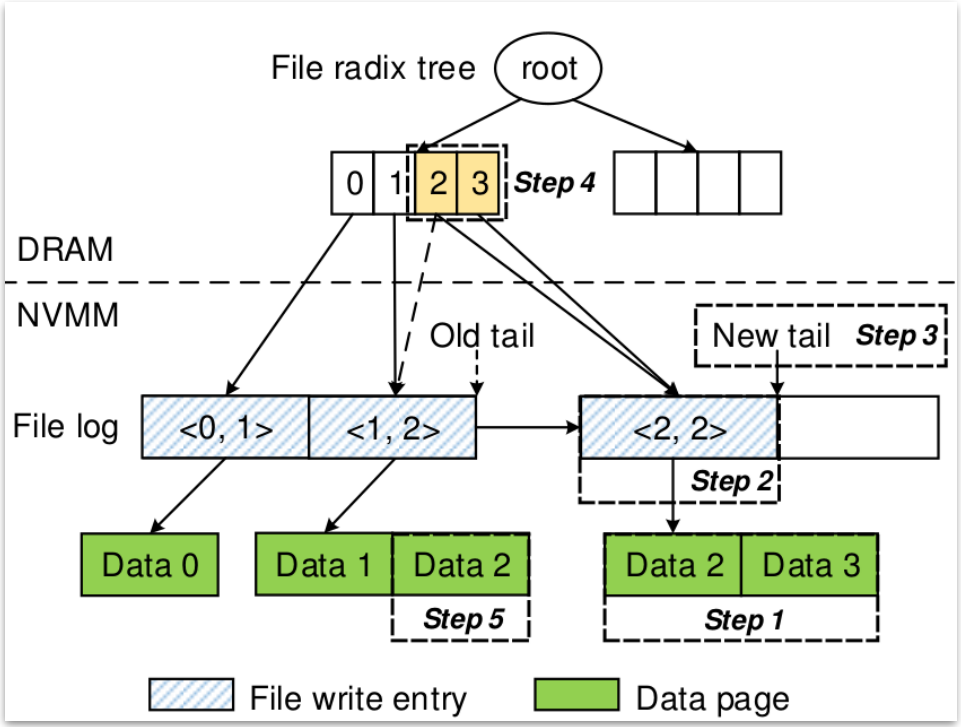


Performance: logs in NVM, and index in DRAM (can be reconstructed)

Smaller log segments (4KB) and implement the log as a link list

Single inode updates (in the inode log), multiple inodes (uses the journal, 64B entries)

Nova : Write Example



We are modifying Data2 and add Data3
`<write order, number of page affected>`

Step 1: find and copy blocks which are to be updated (Copy-on-write)

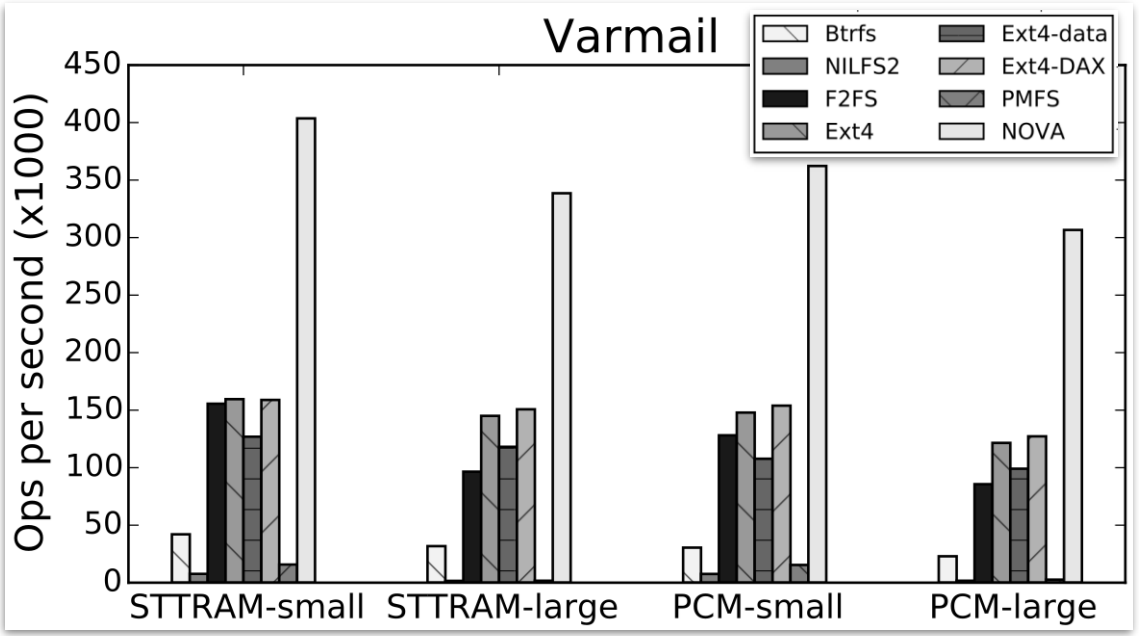
Step 2: Add to the file inode log

Step 3: Update the log tail pointer (after this the write is it durable)

Step 4: Update the DRAM index for fast lookup

Step 5: Garbage collect old pages

Nova Performance Comparison



For varmail (this workload) : 3.1-216x outperforms other file systems

Summary

How hardware influences file system design

- Hardware details of NAND flash SSDs
- SSD-aware file systems
- Changes brought about by NVM
- NVM-aware file systems